THE OPEN SOURCE AUDIT

A Capstone Project

Open Source Software | VITyarthi

Field	Details
Student Name	Pranav
Software Chosen	Python
Course	Open Source Software (OSS NGMC)
OS Used	Ubuntu 24.04.4 LTS
Kernel	6.17.0-19-generic
Date of Submission	March 2026

"Every line of code you write will sit on top of open-source foundations."

Introduction

When I first started learning to code, Python was just a tool — something I typed into to get answers out. I didn't think much about where it came from, who built it, or why it was free for anyone in the world to download and use without paying a single rupee. That changed when I sat down to actually audit it for this project.

Python is one of the most used programming languages on the planet right now. It runs machine learning models at Google, powers Instagram's backend, drives NASA's scientific computing pipelines, and is probably the first language millions of students learn when they open a laptop for the first time. And all of that sits on top of a project that was started by one Dutch programmer in 1991 who just wanted something cleaner and more readable than what existed at the time.

This report is my attempt to understand Python not just as a language, but as an open-source project — its origin story, the license that keeps it free, the philosophy behind it, how it lives on a Linux system, the ecosystem it has built around itself, and how it compares to the proprietary world it often has to compete with.

I also wrote five shell scripts as part of this project, which I ran on Ubuntu 24.04.4 LTS. Those scripts are documented in the later sections of this report, along with what I observed when I ran them.

Part A — Origin and Philosophy

A1. The Problem Python Was Created to Solve

To understand why Python exists, you have to go back to the late 1980s. Guido van Rossum was a programmer at CWI (Centrum Wiskunde & Informatica) in the Netherlands, working on a language called ABC. ABC was actually a well-designed teaching language — clean, readable, high-level — but it had serious problems when it came to doing real work. It couldn't be extended easily, it had no concept of file I/O in the way Unix programmers expected, and it was completely closed off from the broader ecosystem of C libraries and system tools that every serious programmer relied on.

Meanwhile, the language most people used for scripting on Unix systems was either shell script (messy, fragile) or Perl (powerful but famously hard to read — there's a joke that Perl code looks like it was written by a cat walking across a keyboard). The tools that existed were either too low-level or too ugly. There was no middle ground: something expressive enough for humans to write quickly, but serious enough to actually connect to the systems underneath.

Guido started writing Python over the Christmas holidays of 1989 — literally just as a personal project to keep himself occupied. His goal was straightforward: take what was good about ABC, fix what was broken, and add the ability to talk to the C world underneath. He wanted a language where the code looked like something a human would actually write. Hence the famous emphasis on indentation and readability — Python forces you to write clean code by making the whitespace itself part of the syntax.

By 1991, Guido released Python 0.9.0 to a Usenet newsgroup. He wasn't trying to start a movement. He just wanted feedback. What he got instead was a community — people who had been looking for exactly this thing and didn't know it until they saw it. The language grew not because a company pushed it, but because real programmers found it useful and started contributing.

The honest answer to 'what problem did Python solve' is: it made programming feel human. Before Python, you had to choose between elegance and power. Python argued you didn't have to. That turned out to be a radical idea.

A2. The License — What It Actually Says

Python uses the PSF License — the Python Software Foundation License. The current version is PSF 2.0, which is compatible with GPL version 3. It's a permissive open-source license, meaning it imposes very few restrictions on what you can do with the software.

The Four Freedoms of Free Software

Richard Stallman defined four freedoms that any truly free software must grant:

- Freedom 0: The freedom to run the program for any purpose.
- Freedom 1: The freedom to study how the program works and change it.
- Freedom 2: The freedom to redistribute copies.
- Freedom 3: The freedom to distribute modified versions.

Python's PSF license grants all four. You can download CPython (the reference implementation), read every line of the interpreter's source code on GitHub, modify it, and distribute your own version. Companies like Anaconda, PyPy, and countless others have done exactly this.

Can a Company Take Python, Modify It, and Sell It?

Yes — and this is what makes the PSF license different from the GPL. Under the GPL, if you distribute a modified version of the software, you must also release your modifications under the GPL. The PSF license does not require this. A company can take CPython, make changes, and keep those changes private — as long as they retain the copyright notice and license text.

This is why you see commercial products built on Python all the time. Palantir's software runs on Python. Bloomberg's financial systems use it. None of them are required to publish their proprietary code. The tradeoff Guido and the PSF made was: we want maximum adoption, even if it means some companies benefit without giving back the code. The bet was that a huge user base would produce more community contributions than a stricter license would.

GPL vs MIT vs PSF — Which Would I Choose?

The GPL is a 'copyleft' license. It says: you can use this freely, but if you distribute a modified version, you must keep it free. It protects the software's freedom by requiring everyone downstream to play by the same rules. The MIT license is the most permissive of all — essentially 'do whatever you want, just keep my name on it.' The PSF falls somewhere in between, closer to MIT in practical terms.

If I were building something I wanted maximum people to use and build on — the way Python's creators wanted — I would choose MIT or PSF. If I were building something where I was worried about a corporation taking my work, making it proprietary, and selling it back to people, I'd choose GPL. For a personal library or a tool I wanted the whole ecosystem to absorb freely, MIT. For something politically important — like an operating system or a compiler — GPL, because you want to ensure it can never be locked away.

Free as in Beer vs Free as in Freedom

This distinction matters more than it sounds. 'Free as in free beer' means zero cost — you don't pay for it. 'Free as in freedom' means you have the right to use, study, modify, and share it.

Python is both, but the freedom part is what makes it important. Consider this: a company could, theoretically, make Python free to download but refuse to release the source code. That would be

free beer, not free speech. The PSF license ensures it's always the second kind — the source is always available, always modifiable. That's why Python has been ported to every conceivable platform, embedded in devices, and extended in ways Guido never imagined. The freedom to look inside and change things is what made that possible.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for: knowledge compounds. When one person solves a problem and publishes the solution, a thousand people don't have to solve it again. The entire modern internet is built on this — Linux, Apache, Python, MySQL, Git. If any one of those had been proprietary, the cost of building what we have today would have been orders of magnitude higher. Open source democratizes access to tools. A student in a small town in India can learn with the same tools that engineers at Google use. That is genuinely remarkable.

The case against: some software is deeply expensive to build. Pharmaceutical software that models drug interactions, real-time safety systems for aircraft, custom enterprise tools — these require sustained investment that open-source models struggle to fund. If everything were open source, you'd have to solve the economic problem of who pays for that work. The answer isn't obvious. Volunteers can't maintain everything indefinitely, and the burnout rate among open-source maintainers is a real documented problem.

My conclusion: the default should lean strongly toward openness, especially for tools that form shared infrastructure. But the question of how to sustainably fund that work is one the community still hasn't fully answered.

Is It Ethical to Build Profits on Free Community Labor?

This is the uncomfortable question the industry mostly avoids. Companies like Google, Meta, Amazon, and Microsoft have collectively extracted enormous commercial value from Linux, Python, and other open-source projects while contributing back unevenly. Some contribute a lot — Google employs Python core developers and has contributed millions of lines of code. Others take far more than they give.

The community-built tools these companies depend on are often maintained by a small number of unpaid or underpaid volunteers. When those maintainers burn out and disappear — which happens — critical infrastructure gets left vulnerable. The Log4j vulnerability in 2021 was a wake-up call: one of the most widely deployed pieces of software in the world was maintained by essentially two volunteers in their spare time. Every major tech company was using it. Almost none were paying for it.

I think it is ethically questionable to profit massively from community labor without contributing back proportionally. It's not illegal. But it's not right either. The open-source model works on a kind

of social contract — take what you need, give back what you can. When large, profitable corporations violate that contract at scale, it erodes the foundation everyone depends on.

Standing on the Shoulders of Giants

In software, almost nothing is built from scratch. Every Python program I've written stands on CPython, which stands on C, which stands on decades of compiler research, which stands on mathematical foundations going back to Turing and Lovelace. Open source makes this inheritance visible and explicit — you can follow the chain all the way down.

Does this undermine original innovation? I don't think so. The freedom to build on what exists without starting over every time is what allows innovation to compound. The people who built TensorFlow didn't have to invent Python first. The people who built Django didn't have to invent HTTP. The stack is shared so that each generation can contribute something genuinely new rather than repeating what was already done.

Part B — Linux Footprint

For this section, I installed and ran Python on my own machine running Ubuntu 24.04.4 LTS. Everything documented here is from actual observation on that system.

Installation Method

On Ubuntu and Debian-based systems, Python 3 is available directly through the APT package manager. The version I installed was Python 3.12.3, which was packaged as part of Ubuntu's official repositories. Installation requires no compilation from source — it is a straightforward:

```
sudo apt install python3
```

The package is maintained by Canonical (Ubuntu's publisher) and the Debian Python maintainers, who track the upstream CPython releases and package them for the distribution. For newer versions not yet in the repository, you can use a PPA (Personal Package Archive) or compile from source using the official CPython repository on GitHub.

Key Directories

After installation, Python distributes itself across several standard Linux directories:

Path	Contents
/usr/bin/python3	The main Python 3 executable
/usr/lib/python3/	Standard library modules
/usr/lib/python3.12/	Version-specific standard library
/usr/include/python3.12/	Header files for C extensions
/usr/share/doc/python3/	Documentation files
/etc/python3/	System-wide Python configuration

I verified the Python library directory exists on my system and confirmed it is owned by root:

```
drwxr-xr-x 3 root root 4096 Aug 5 2025 /usr/lib/python3
```

User and Group

Python itself runs as whatever user invokes it — there is no dedicated system user for Python the way there is for a web server like Apache (which typically runs as www-data). When I run a Python script, it runs as me: lucifer on my machine. This matters for security because Python scripts inherit all the permissions of the user who invokes them. A Python script run as root has root privileges, which is why security best practices discourage running Python applications as root.

For production deployments — say, a Python web application running with Gunicorn — the standard practice is to create a dedicated low-privilege user account and run the application under

that account, with only the permissions it strictly needs. This principle of least privilege is a core part of Linux security.

Service Management

Python itself is not a service — it's an interpreter. But Python applications can be run as services. On a modern Ubuntu system using systemd, a Python application can be managed like this:

```
# Create a service file at /etc/systemd/system/myapp.service
# Start the service
sudo systemctl start myapp
# Stop the service
sudo systemctl stop myapp
# Check status
sudo systemctl status myapp
# Enable on boot
sudo systemctl enable myapp
```

Update Model

Python receives security patches through two channels on Ubuntu. First, Canonical backports security fixes from newer CPython versions into the packaged version on the system — so even if you're running Python 3.12.3, a security fix from 3.12.5 may be silently applied as a patch. These updates arrive via the standard APT security repository and are applied with:

```
sudo apt update && sudo apt upgrade
```

Second, for users who need the absolute latest Python version, the PSF maintains python.org and publishes new releases directly. The community's response to vulnerabilities is typically fast — CVEs are patched and published within days of disclosure, and the transparency of open source means the fix can be independently verified by anyone.

Part C — The FOSS Ecosystem

What Python Depends On

CPython — the standard Python implementation — is written in C. At the lowest level, it depends on the GNU C Library (glibc), which is a foundational piece of the Linux stack. The Python build system uses GNU Make. The standard library includes bindings to OpenSSL (for cryptography), zlib (for compression), sqlite3 (for databases), and readline (for interactive shell features). None of these are Python's own work — they are open-source C libraries that Python wraps and exposes to Python programmers.

This dependency chain means that when a vulnerability is found in OpenSSL, Python distributions that use it are potentially affected. The open-source nature of the entire stack means these vulnerabilities can be found, disclosed, and fixed in public — something that's much harder in a closed system.

What Python Has Enabled

The ecosystem Python has generated is almost impossible to fully catalogue. Some of the most significant:

- Django and Flask — web frameworks that have powered thousands of startups and large applications. Instagram was built on Django.
- NumPy and Pandas — scientific computing libraries that made Python the default language of data science, displacing MATLAB in many research contexts.
- TensorFlow and PyTorch — deep learning frameworks from Google and Meta respectively, both primarily Python-facing. The AI revolution of the past decade largely runs through Python.
- Jupyter Notebooks — an interactive computing environment that changed how scientists and analysts work with data. Born from IPython, another Python project.
- Ansible — IT automation tool written in Python, used by sysadmins worldwide.

Each of these is itself an open-source project with its own ecosystem, community, and dependents. Python didn't just spawn tools — it spawned entire sub-communities.

The Community and Governance

Python is governed by the Python Software Foundation (PSF), a US non-profit that holds the intellectual property rights, funds development, and organises PyCon — the annual Python conference that draws tens of thousands of attendees globally. Day-to-day language decisions are made by a Steering Council of five elected core developers, a governance structure adopted after Guido van Rossum stepped back from his role as 'Benevolent Dictator For Life' (BDFL) in 2018.

For day-to-day discussion, the community uses Python Discourse (discuss.python.org), the python-dev mailing list, and GitHub pull requests on the CPython repository. Language changes go through a formal PEP (Python Enhancement Proposal) process — a numbered document that proposes a change, argues for it, and records the community decision. PEP 8, for instance, is the style guide that defines how Python code should be written. PEP 20 is 'The Zen of Python' — 19 aphorisms about the philosophy of the language.

Part D — Open Source vs Proprietary

Python's closest proprietary competitor in the general-purpose scripting and data space would historically be MATLAB for scientific computing and, more recently, certain enterprise data platforms. In the web space, it competes with platforms like Node.js (also open source) and older proprietary stacks like classic ASP or ColdFusion.

For this comparison, I'll use MATLAB as the proprietary reference point, since it serves a genuinely similar audience in scientific and engineering computing.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free. Always. For everyone.	Expensive commercial license. Student editions subsidised but not free.
Security	Source code is public. Anyone can audit it, find bugs, and report them.	Closed source. Security depends entirely on MathWorks' internal process.
Support	Community forums, Stack Overflow, official docs. No guaranteed SLA.	Official MathWorks support with paid licenses. More predictable for enterprises.
Freedom to Modify	Full. You can fork CPython and build your own interpreter.	None. You use what MathWorks gives you.
Community vs Corporate Control	PSF and elected Steering Council. Community-driven.	Fully controlled by MathWorks. Roadmap is internal.
Ecosystem Size	Enormous. Hundreds of thousands of packages on PyPI.	Toolboxes available but much smaller and mostly paid.

Overall Verdict

For almost any use case I can imagine — web development, data science, automation, machine learning, scientific computing, scripting — Python is the better choice. The combination of zero cost, massive ecosystem, and genuine freedom to inspect and modify the tools you depend on is simply hard to match. The one area where MATLAB has a legitimate advantage is in toolboxes that have no Python equivalent (some very specialised signal processing or control systems domains) and in environments where institutional support and guaranteed reliability are more important than cost (certain regulated industries, for instance).

Would I recommend Python for real-world deployment? Without hesitation. Would I contribute to it? I've already filed a minor documentation issue on the CPython GitHub as part of this project. That counts, I think.

Part E — Shell Script Documentation

All five scripts were written and tested on Ubuntu 24.04.4 LTS with kernel 6.17.0-19-generic, logged in as user 'lucifer' in the ~/oss-audit directory.

Script 1 — System Identity Report

This script acts like a welcome screen for the system. It collects information about the Linux environment — the distribution name, kernel version, current user, uptime, and date — and prints it in a formatted report. It also prints the license that governs the OS (GNU GPL) and Python (PSF License).

Concepts Used

- Variables: name, software, kernel, user, uptime, date_now, distro
- Command substitution with $(): uname -r, whoami, uptime -p, date, grep + cut for distro name
- echo for formatted output

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Pranav | Project: OSS Audit (Python)

name="Pranav"
software="Python"
kernel=$(uname -r)
user=$(whoami)
uptime=$(uptime -p)
date_now=$(date)
distro=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')

echo "------------------------------------"
echo " Open Source Audit Report"
echo "Name       : $name"
echo "Software   : $software"
echo "Distro     : $distro"
echo "Kernel     : $kernel"
echo "User       : $user"
echo "Uptime     : $uptime"
echo "Date       : $date_now"
echo "OS License: GNU GPL"
echo "Python License: PSF License"
echo "------------------------------------"
```

Output Observed

```
Open Source Audit Report
Name       : Pranav
Software   : Python
Distro     : Ubuntu 24.04.4 LTS
Kernel     : 6.17.0-19-generic
```

```
User       : lucifer
Uptime     : up 17 minutes
Date       : Tue Mar 24 12:13:31 AM IST 2026
OS License: GNU GPL
Python License: PSF License
```

The script ran without errors. The distro detection using grep on /etc/os-release is the standard way to identify the distribution in a distribution-agnostic script, since it works on virtually all modern Linux systems.

Script 2 — FOSS Package Inspector

This script checks whether python3 is installed, finds its version and path using command -v and --version, then queries dpkg for package manager information. A case statement is used to print a description of Python's purpose based on the package name.

Concepts Used

- if-then-else: checks if the package command exists
- command -v: portable way to check if a program is in PATH
- case statement: prints description based on package name
- dpkg -l with grep and head: lists installed packages

```
#!/bin/bash
# Script 2: Package Inspector
# Author: Pranav

pkg="python3"

if command -v $pkg &> /dev/null; then
    echo "Status   : Installed"
    echo "Version  : $($pkg --version)"
    echo "Path     : $(which $pkg)"
else
    echo "Status   : Not Installed"
fi

dpkg -l | grep $pkg | head -5

case $pkg in
    python3) echo "Python: community-driven programming language" ;;
    *) echo "Open-source tool" ;;
esac
```

Output Observed

```
Status   : Installed
Version  : Python 3.12.3
Path     : /usr/bin/python3
ii  python3   3.12.3-0ubuntu2.1   amd64   ...
```

```
Python: community-driven programming language
```

Script 3 — Disk and Permission Auditor

This script loops through five key system directories and reports the permissions, ownership, and disk usage of each. It then checks specifically for Python's library directory and prints its details.

Concepts Used

- Array: dirs=(...) holding directory paths
- for loop: iterates over the array
- [-d]: conditional check if directory exists
- ls -ld + awk: extracts permissions and owner fields
- du -sh + cut: extracts human-readable size

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
dirs=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
for d in "${dirs[@]}"
do
    if [ -d "$d" ]; then
        perm=$(ls -ld $d | awk '{print $1, $3, $4}')
        size=$(du -sh $d 2>/dev/null | cut -f1)
        echo "$d => $perm | Size: $size"
    fi
done

if [ -d "/usr/lib/python3" ]; then
    echo "Python directory exists"
    ls -ld /usr/lib/python3
fi
```

Output Observed

```
/etc => drwxr-xr-x root root | Size: 12M
/var/log => drwxrwxr-x root syslog | Size: 385M
/home => drwxr-xr-x root root | Size: 601M
/usr/bin => drwxr-xr-x root root | Size: 238M
/tmp => drwxrwxrwt root root | Size: 92K
Python directory exists
drwxr-xr-x 3 root root 4096 Aug 5 2025 /usr/lib/python3
```

Interesting observation: /tmp has the sticky bit set (the 't' at the end of drwxrwxrwt). This means that even though anyone can write to /tmp, users can only delete their own files — not files created by other users. This is a standard Linux security measure for shared temporary directories.

Script 4 — Log File Analyzer

This script reads a log file line by line using a while-read loop, counts how many lines contain a given keyword (passed as a command-line argument, defaulting to 'error'), and prints a summary along with the last five matching lines.

Concepts Used

- Command-line arguments: $1 for log file path, $2 for keyword
- [! -f]: file existence check
- while read loop: reads log file line by line
- if-then with grep -iq: case-insensitive keyword matching
- Counter variable: incremented with $((count+1))
- grep + tail: prints last 5 matching lines

```
#!/bin/bash
# Script 4: Log File Analyzer
logfile=$1
keyword=${2:-error}
count=0

if [ ! -f "$logfile" ]; then
    echo "File not found!"
    exit 1
fi

while read line
do
    if echo "$line" | grep -iq "$keyword"; then
        count=$((count+1))
    fi
done < "$logfile"

echo "Keyword '$keyword' found $count times"
grep -i "$keyword" "$logfile" | tail -5
```

Output Observed

```
Keyword 'error' found 108 times

Last 5 matching lines:
2026-03-24T00:12:06 ... Error creating IO channel ...
2026-03-24T00:12:46 ... Error creating IO channel ...
2026-03-24T00:13:46 ... Error creating IO channel ...
2026-03-24T00:15:06 ... Error creating IO channel ...
2026-03-24T00:17:01 ... Error creating IO channel ...
```

The 108 error entries in the syslog were all 'Error creating IO channel for /proc/self/mountinfo: Invalid argument' from the tracker-miner-fs process. This is a known benign warning on Ubuntu systems with certain GNOME tracker configurations — not a real system problem. This is a useful example of why log analysis matters: a raw count of 108 errors sounds alarming until you look at what the errors actually are.

Script 5 — Open Source Manifesto Generator

This is the most personal script. It asks three interactive questions using read, then constructs a short personalised statement about open-source values and saves it to a text file.

Concepts Used

- read -p: interactive user input with prompt
- String concatenation: building sentences from variables
- File writing: using > (overwrite) and >> (append)
- date command: stamping the manifesto with the current date
- cat: displaying the final file contents

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

read -p "Tool you use daily: " tool
read -p "Freedom means: " freedom
read -p "What will you build: " build

date_now=$(date '+%d %B %Y')
file="manifesto_pranav.txt"

echo "My Open Source Manifesto" > $file
echo "Date: $date_now" >> $file
echo "I use $tool every day, which is built by the open-source community." >>
$file
echo "For me, freedom means $freedom." >> $file
echo "One day, I will build $build and share it freely." >> $file

cat $file
```

The manifesto output is saved to manifesto_pranav.txt. This script demonstrates how shell scripts can interact with users, process their input as variables, and produce a meaningful personalised output from just a few lines of code — which is itself a demonstration of what makes scripting powerful.

Conclusion

Writing this report changed the way I think about the tools I use every day. Before this project, Python was just something I installed and used. Now I see it differently — as a 35-year-old collaborative project, shaped by thousands of contributors, governed by an elected council, licensed in a way that was deliberately designed to maximise adoption without compromising freedom.

The five shell scripts forced me to actually interact with Python as a Linux citizen — to look at where it lives on the filesystem, to inspect its packages, to read the logs my system generates. That practical work made the philosophical questions feel less abstract. When you see that /usr/lib/python3 is owned by root and has permissions drwxr-xr-x, you understand why the Linux permission model matters. When you run a log analyzer and find 108 errors that turn out to be completely harmless, you understand why understanding your system matters more than just counting numbers.

The ethical questions this project asked — about corporate extraction of community labor, about whether open source is always better, about what freedom in software actually means — don't have clean answers. But I think the questions themselves are important. Every software engineer will make choices about what to build, how to license it, what to open up and what to keep closed. Understanding the history and philosophy of open source is part of being equipped to make those choices thoughtfully.

I'll keep using Python. But now I'm also paying attention to the PSF's funding calls, the GitHub issues on CPython, and the PEPs being debated right now. That feels like the right response to what this project asked of me.

References

- Python Software Foundation. PSF License Agreement. https://docs.python.org/3/license.html
- GNU Project. The Free Software Definition. https://gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition. https://opensource.org/osd
- Guido van Rossum. A Brief Timeline of Python. https://python-history.blogspot.com
- Eric S. Raymond. The Cathedral and the Bazaar. https://catb.org/~esr/writings/cathedral-bazaar
- CPython Source Repository. https://github.com/python/cpython
- Ubuntu Package Archive. python3 3.12.3-0ubuntu2.1
- SPDX License List. PSF-2.0. https://spdx.org/licenses/PSF-2.0.html
- William Shotts. The Linux Command Line. https://linuxcommand.org
- Python Enhancement Proposals Index. https://peps.python.org